Actividad 5: Informe de conclusiones

1. Pregunta de negocio y enfoque elegido

La pregunta que buscamos responder es: ¿Qué clientes sostienen el negocio y cuáles se están enfriando? El objetivo es construir una segmentación que permita decidir sobre quién actuar primero.

Utilizamos el análisis RFM, que nos permite ver y estudiar cómo se comportan los clientes a partir de tres aspectos:

Recencia: nos indica el tiempo transcurrido desde el último alquiler del cliente hasta la fecha máxima de alquiler registrada en la base. Se mide contra la base y no contra la fecha de hoy, porque los datos de la base corresponden a un período pasado: si se midiera contra hoy, todos los clientes parecerían inactivos y no se distinguiría a nadie.

Frecuencia: representa la cantidad de alquileres realizados.

Valor monetario: monto total de los pagos asociados a sus alquileres.

Cada dimensión se puntúa de 1 a 5 por quintiles, y las tres puntuaciones se combinan en un segmento de tres cifras. Por ejemplo, 555 sería un cliente campeón y 111 un cliente perdido.

Elegimos RFM y no ordenar solo por monto gastado porque el monto, por sí solo, no detecta a los clientes que se están enfriando: alguien que gastó mucho hace tiempo y no volvió parece valioso, pero ya no sostiene el negocio. Al sumar la recencia y la frecuencia podemos diferenciar a los clientes por su actividad y por el valor que tienen para el negocio, e identificar tanto a los más activos como a los que llevan más tiempo sin alquilar. Esto organiza la información en segmentos que facilitan la toma de decisiones: priorizar acciones de retención en los clientes que generan mayor valor y evaluar estrategias para recuperar a los que muestran señales de menor actividad.

2. Decisiones de diseño

Reparto de trabajo entre SQL y Python

SQL Server une las tablas de clientes, alquileres y pagos, y resume todo por cliente (fecha del último alquiler, cantidad de alquileres y total pagado). De esa forma, la base entrega una sola fila por cliente (599) en lugar de miles de filas de detalle. Python toma ese resultado, lo combina con el archivo plano complementario (que no está en la base), calcula los quintiles y el segmento, hace los gráficos y carga el resultado final. Elegimos este reparto porque el motor está pensado para agrupar y sumar grandes volúmenes de datos, mientras que Python resuelve lo que SQL no tiene a mano: cruzar un archivo externo, graficar y registrar cada paso del proceso.

Para verificar el cálculo, la misma métrica se reconstruyó en pandas a partir del detalle crudo y se comparó con la de SQL. Los resultados coinciden para los 599 clientes.

Criterios de limpieza

- Los clientes sin alquileres no se descartan: su monto es 0 y su recencia se completa con el valor más alto más uno, para que queden en el grupo menos reciente. En esta base no hubo ningún caso.
- Los empates en el orden se desempatan por el identificador del cliente, para que el resultado sea siempre el mismo.
- Los valores atípicos de monto se detectaron con el criterio IQR. Son 6 clientes y se conservaron: son clientes de alto valor plausibles, no errores.
- No se inventaron ni reconstruyeron datos. La base migrada tiene 5 pagos menos que el volcado original (16.044 contra 16.049). Se dejó la diferencia a la vista: es consistente con

pagos sin alquiler que no se migraron, pero no se puede confirmar porque esas filas no están en SQL Server.

Controlador de conexión

Usamos pyodbc con el controlador ODBC Driver 18 for SQL Server, a través de SQLAlchemy. Se conecta con la autenticación integrada de Windows, así que no hay usuario ni contraseña escritos en el código. El servidor, la base y el controlador se pueden cambiar con variables de entorno, sin modificar el código.

Estrategia de carga

El pipeline extrae, transforma y carga en tres etapas separadas. La carga se hace dentro de una sola transacción: crea la tabla destino si no existe, borra las filas anteriores e inserta el resultado nuevo. Si algo falla, se deshace todo. Así, al ejecutarlo dos veces seguidas no duplica datos, porque cada ejecución reemplaza la foto anterior. Antes de cargar se controla que no haya clientes duplicados y después se verifica que la cantidad de filas del destino coincida con la esperada. Cada paso queda registrado en un log y, si hay un error, el pipeline termina con código de error en lugar de silenciarlo.

3. Síntesis del diagnóstico de calidad y su impacto

Diagnóstico de calidad

Se revisó la forma del conjunto (599 clientes), los tipos de dato, los nulos, los duplicados por clave, los estadísticos con percentiles y los valores atípicos (IQR). Además se aplicaron los controles específicos de la variante: pagos sin alquiler asociado, clientes sin ningún pago, pagos negativos y pagos asociados a otro cliente. Cada control se calculó en SQL Server y en pandas, y los resultados coincidieron. También se verificaron dos reglas de negocio: no se admiten pagos negativos y la recencia no puede ser negativa.

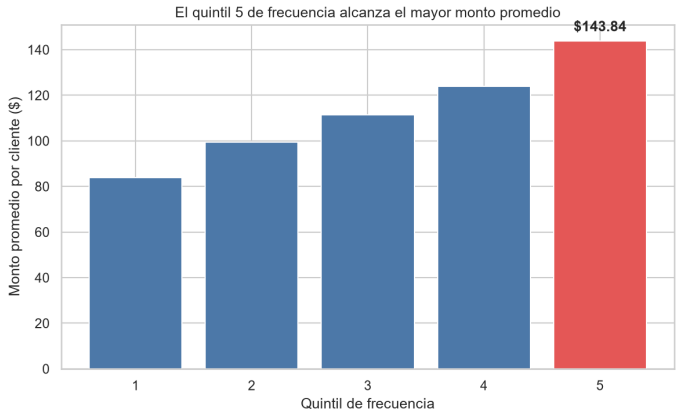
Resultados: no hay nulos ni clientes duplicados. No aparece el defecto conocido de Sakila: no hay pagos con la clave de alquiler en NULL, ni clientes sin pagos, ni pagos negativos. Hay 6 clientes con montos atípicos; se conservaron porque son clientes de alto valor plausibles, no errores. En la reconciliación con el origen, 15 de las 16 tablas coinciden; payment tiene 16.044 filas contra 16.049 del original (5 menos).

Impacto sobre las conclusiones

- Como los datos no tenían nulos, duplicados ni valores inválidos, el RFM se calculó sobre los 599 clientes.
- Los 5 pagos faltantes no se pueden recuperar. La diferencia es consistente con pagos sin alquiler que no se migraron y, en ese caso, no afectan al RFM; pero no podemos confirmarlo porque esas filas no están en SQL Server.
- La recencia está muy concentrada: la mediana es de 175 días y al menos una cuarta parte de los clientes tiene recencia 0. Esto genera empates, y el puntaje de recencia depende en buena parte del criterio de desempate. Por eso las conclusiones sobre quiénes se están enfriando deben tomarse con cuidado.

4. Hallazgos

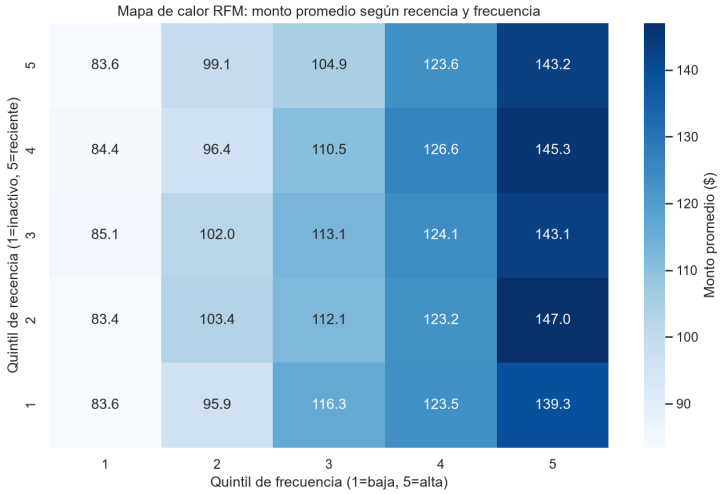
Hallazgo 1: los clientes más frecuentes son los que más aportan



En el gráfico se ve que el monto promedio va subiendo en cada quintil de frecuencia. En el quintil 1 es de unos $84 y en el quintil 5 llega a $143,84, o sea, alrededor de un 70 % más. Esto es de esperar, porque el monto es la suma de los pagos de los alquileres, pero igual muestra lo mucho que pesa la frecuencia.

Decisión: conviene cuidar y retener primero a los clientes que más alquilan, por ejemplo con beneficios por seguir siendo clientes.

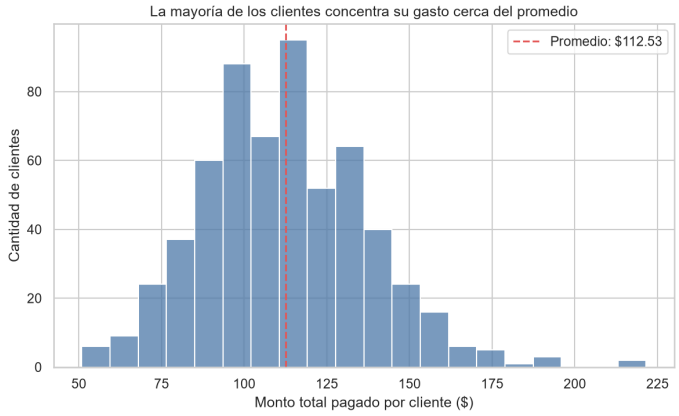
Hallazgo 2: la recencia casi no distingue el valor de un cliente



En el mapa de calor el color cambia de izquierda a derecha, que es la frecuencia, pero casi no cambia de arriba hacia abajo, que es la recencia. Por ejemplo, en el quintil 1 de frecuencia el monto promedio va de $83,4 a $85,1 según la recencia, y en el quintil 5 va de $139,3 a $147,0. Esto tiene que ver con el diagnóstico de calidad: la recencia está muy concentrada y hay muchos empates.

Decisión: no conviene usar solo la recencia para decidir a quién reactivar. Hay que mirarla junto con la frecuencia y el monto, y tomar con cuidado la lista de clientes que se están enfriando.

Hallazgo 3: el negocio no depende de unos pocos clientes grandes



El monto total de la mayoría de los clientes está cerca del promedio, que es de $112,53. El 90 % de los clientes está entre $73,80 y $155,60, y solo 6 superan el límite del IQR. Es decir, no hay un grupo chico de clientes enormes que sostenga todos los ingresos.

Decisión: las acciones de retención tienen que apuntar a muchos clientes y no solo a unos pocos. Los 19 clientes del segmento 555 se podrían usar como grupo de prueba para los beneficios, sabiendo que quiénes entran depende del criterio de desempate.

5. Limitaciones

Esto es lo que no se puede afirmar con estos datos y por qué.

- Es un análisis descriptivo. No podemos decir que un cliente con puntaje bajo "se fue" ni que una campaña lo va a recuperar, porque el RFM cuenta lo que pasó: no explica por qué ni predice lo que va a pasar. Tampoco hay datos de costos, de rentabilidad ni de qué películas prefiere cada cliente, así que no sabemos qué segmento conviene más ni cuánto costaría actuar sobre él.
- La recencia distingue poco entre clientes. La mediana es de 175 días y al menos 151 de los 599 clientes tienen recencia 0. Un quintil tiene unos 120 clientes, entonces clientes con exactamente el mismo valor terminan en grupos distintos, y eso se decide con el desempate por identificador. Por eso no podemos asegurar que dos clientes iguales tengan el mismo puntaje, ni que los 19 del segmento 555 sean los únicos "mejores". Además, el mapa de calor muestra que el monto casi no cambia según la recencia.
- No podemos confirmar que un cliente esté "perdido". La recencia máxima es de 181 días, unos seis meses. Es poco tiempo para decidir cuánto sin alquilar significa que se fue: puede haber dejado de alquilar por motivos que no vemos.
- Los quintiles son relativos. Un 5 muestra la posición del cliente dentro de esta base, no un gasto alto en términos absolutos, y siempre va a haber cerca de un 20 % de clientes en cada grupo. Por eso no se pueden comparar con otra base ni con otro período. Además, qcut y NTILE coinciden en el 99,7 % de los casos: hay 2 clientes por métrica en el borde que podrían cambiar de segmento según la técnica.
- Frecuencia y monto están relacionados. Como el monto suma los pagos de los alquileres, más alquileres significa más monto. Por eso el hallazgo 1 es esperable, y el RFM repite parte de la información.
- Hay 5 pagos que no pudimos recuperar. El original tiene 16.049 pagos y SQL Server tiene 16.044. Es consistente con pagos sin alquiler que no se migraron, pero no lo podemos confirmar porque esas filas no están en la base. Si no fuera así, el monto de algunos clientes estaría un poco subestimado.

- Los datos son de ejemplo. Sakila es una base de prueba y no son clientes de un negocio real, así que las decisiones son ilustrativas. Además, el archivo plano complementario solo tiene 30 de los 599 clientes (5 %) y es de prueba, por eso no lo usamos para sacar conclusiones.

6. Dificultades encontradas y cómo se resolvieron

- Los nombres de la base están en español. El código no podía depender de esos nombres, así que armamos un diccionario de esquema (esquema.py) que relaciona los nombres de Sakila en inglés con los de nuestra base, y el notebook lo usa para armar las consultas.
- NTILE y qcut no daban exactamente lo mismo. Con 599 clientes, que no es múltiplo de cinco, reparten el resto de forma distinta. En lugar de esconderlo, replicamos en pandas la regla de reparto de NTILE, que coincide al 100 %, y dejamos documentada la diferencia.
- SQL Server y pandas calculaban distinto la recencia. DATEDIFF de SQL Server cuenta días de calendario y no períodos completos de 24 horas. Lo resolvimos normalizando las fechas a medianoche en pandas, y las métricas pasaron a coincidir.
- Muchos clientes empatan en recencia. El quintil no puede partirlos sin un criterio, así que desempatamos por el identificador del cliente para que el resultado sea siempre el mismo, y lo declaramos como limitación.
- Diferencia de 5 pagos con el origen. No inventamos ni reconstruimos filas: dejamos la diferencia a la vista, aplicamos los controles de calidad y la declaramos como consistente con pagos sin alquiler que no se migraron, sin poder confirmarlo.
- Que el pipeline no duplique datos. Cargamos dentro de una transacción que borra e inserta, de modo que cada ejecución reemplaza a la anterior. Los registros del pipeline muestran dos ejecuciones seguidas con 599 filas cargadas y verificadas en cada una.